

EstateIQ

Gurgaon Real Estate ML System

Technical Project Report

Production-Grade Machine Learning · Enterprise Documentation

Machine Learning

FastAPI Backend

Streamlit UI

Docker CI/CD

Random Forest

TF-IDF
Recommender

0.8211

R² Score
Variance Explained

0.5304 Cr

MAE
Mean Absolute Error

1.1727 Cr

RMSE
Root Mean Sq. Error

Version

1.0 — Final Submission

Date

May 2026

Classification

Confidential

TABLE OF CONTENTS

01	Project Problem Statement	...
02	Theory Behind the Project	...
	2.1 Machine Learning Theory	...
	2.2 Recommendation System Theory	...
	2.3 Backend Engineering Theory	...
	2.4 Frontend Theory	...
03	Project Architecture	...
04	Dataset Description	...
05	Data Cleaning and Preprocessing	...
06	Feature Engineering	...
07	Machine Learning Pipeline	...
08	Backend Development	...
09	Frontend Dashboard	...
10	Analytics and Visualization	...
11	DevOps and Deployment	...
12	Testing and Validation	...
13	Results Achieved	...
14	Challenges Faced	...
15	Scalability and Future Improvements	...

01 PROJECT PROBLEM STATEMENT

The real estate sector is one of the most complex and rapidly changing industries, especially in metropolitan regions such as Gurgaon, India. Property prices vary significantly depending on multiple factors including location, built-up area, furnishing status, nearby infrastructure, amenities, property age, floor category, market demand, and accessibility. Determining the actual market value manually becomes highly difficult and often inaccurate due to these continuously changing parameters.

Traditional real estate platforms mainly provide property listings without offering intelligent analytics, predictive systems, or recommendation engines. Buyers, investors, and sellers are therefore forced to manually compare multiple listings and estimate prices based on incomplete or inconsistent information. This process is time-consuming and frequently results in incorrect property valuation decisions.

Core Data Quality Challenges

- **Missing values** and duplicate records across raw datasets
- **Inconsistent price formats:** Lac, Lakh, Cr, Crore with different capitalizations
- **Text-heavy descriptions** and unstructured amenities information
- **Multiple area units:** square feet, yards, meters, acres in the same field
- **Incorrect or noisy location data:** same sector appearing in multiple text formats

Primary Objectives of EstateIQ

- Predict residential property prices accurately using machine learning
- Recommend similar apartment projects based on user preferences
- Provide real-time analytics and market insights through interactive dashboards
- Visualize locality-based pricing trends with geospatial Mapbox maps
- Support scalable production deployment architecture (Docker, Railway, Streamlit Cloud)
- Simulate a real-world enterprise machine learning workflow end-to-end

02 THEORY BEHIND THE PROJECT

2.1 Machine Learning Theory — Supervised Regression

This project is based on **Supervised Machine Learning**, specifically **Regression Analysis**. The model learns patterns from labeled historical property data and predicts continuous price values for unseen inputs. **Input Features = Property Attributes;** **Target Variable = Property Price (Crore).**

Random Forest Regression

Random Forest Regression is an Ensemble Learning Algorithm based on Decision Trees. It combines multiple trees and aggregates outputs for more accurate and stable predictions.

Core Principle	Description
Bootstrap Sampling	Each tree is trained on a random subset of data with replacement, increasing diversity.
Feature Randomization	At each split a random feature subset is considered, reducing tree correlation.
Ensemble Averaging	Final prediction is the mean of all tree outputs, dramatically reducing variance.

Why Random Forest for Real Estate Pricing

- Handles complex **non-linear relationships** without manual polynomial terms
- Works well with **mixed categorical and numerical** tabular inputs
- Resistant to **overfitting** compared to individual decision trees
- Robust against **outliers and noisy data** — common in real estate listings
- Provides **feature importance** scores for business interpretation

2.2 Recommendation System Theory

The recommendation engine implements **Content-Based Filtering** using TF-IDF Vectorization and Cosine Similarity to identify and rank similar apartment projects from a corpus of 247 Gurgaon projects.

Component	Theory / Implementation
TF-IDF Vectorizer	Term Frequency–Inverse Document Frequency converts textual apartment metadata into numerical vectors. stop_words='english', ngram_range=(1,2) captures phrases like 'swimming pool' and 'sector 48'.
Cosine Similarity	Measures the angle between two TF-IDF vectors. Score near 1.0 = highly similar; near 0 = dissimilar. Efficient for sparse high-dimensional text vectors.
Corpus Construction	PropertyName + PropertySubName + NearbyLocations + LocationAdvantages + TopFacilities are concatenated into a single lowercase search_text document per apartment project.

2.3 Backend Engineering Theory

FastAPI was selected for its ASGI-based asynchronous execution, automatic OpenAPI documentation, native Pydantic integration for type-safe request validation, and excellent ML serving performance. `lru_cache(maxsize=1)` ensures the sklearn model and TF-IDF vectorizer are loaded once per process lifecycle and reused on all subsequent requests, minimizing inference latency.

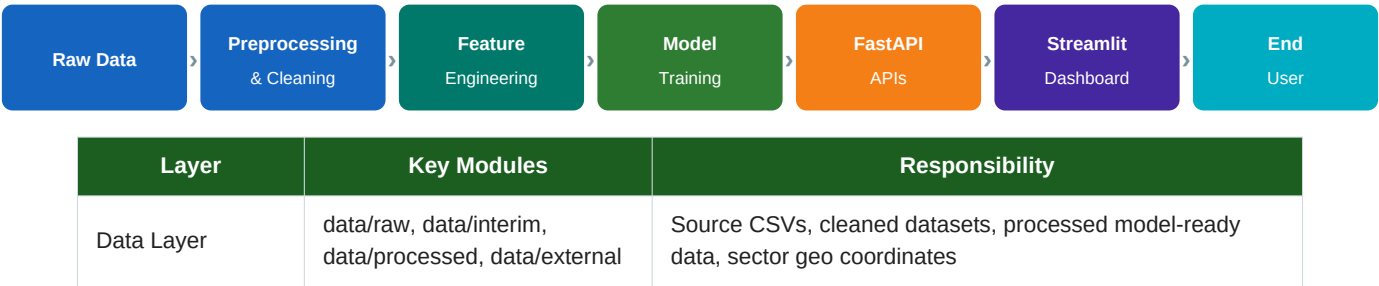
2.4 Frontend Theory

Streamlit enables rapid Python-native dashboard development with built-in support for interactive widgets, session state management, and `@st.cache_data` decorators for efficient data loading. The frontend communicates with the FastAPI backend via HTTP API calls, maintaining a clean separation between UI presentation and ML inference logic.

03 PROJECT ARCHITECTURE

EstateIQ follows a modular, production-grade layered architecture with clear separation of concerns. The `src/` package remains importable independently of Streamlit; the `app/` package wraps `src/` into user-facing services, API routes, pages, and UI components.

End-to-End Machine Learning Workflow



ML Layer	src/data, src/features, src/models, src/pipelines	Preprocessing, feature engineering, training, inference pipeline, analytics
Backend Layer	app/api.py, app/prediction, app/recommender	FastAPI REST endpoints, cached model service, cached recommender service
Frontend Layer	app/streamlit_app.py, app/pages, app/components	Streamlit pages, UI components, Plotly chart factories, HTTP API client
Deployment Layer	deployment/docker, deployment/ci-cd, deployment/aws	Dockerfile, docker-compose, GitHub Actions CI, Railway, AWS documentation

04 DATASET DESCRIPTION

Three raw datasets are collected from Gurgaon property listing portals and combined into a unified 3,554-row model-ready dataset through a multi-stage cleaning and engineering pipeline.

File	Rows	Cols	Description
data/raw/flats.csv	3,017	20	Raw flat/apartment listing data from portals
data/raw/houses.csv	1,044	21	Raw independent house/villa listing data
data/raw/apartments.csv	247	7	Apartment project metadata for recommendations
data/interim/..._cleaned_v2.csv	3,803	23	Feature-engineered with luxury scores
data/interim/..._outlier_treated.csv	3,555	24	Outlier-treated with area_room_ratio
data/processed/..._imputation.csv	3,554	18	After missing value imputation
data/processed/..._feature_selection_v2.csv	3,554	13	Final model-ready training dataset
data/external/latlong.csv	129	2	Sector coordinates for Mapbox visualisations

05 DATA CLEANING AND PREPROCESSING

Raw real estate data contains significant noise, inconsistencies, and formatting variations. The preprocessing pipeline in `src/data/preprocessing.py` implements systematic cleaning across five structured stages.

Step 1	Price Normalization <code>treat_price()</code> Converts price strings ('85 Lac', '1.5 Cr', '2 Crore') to float crore values. Handles missing values, empty strings, and 'Price on Request' entries. Lac/Lakh values are divided by 100.
Step 2	Sector Normalization <code>normalize_sector()</code> Lowercases raw sector/address text, removes 'gurgaon' and 'gurugram' tokens, collapses whitespace. Reduces categorical fragmentation from the same locality appearing in multiple formats.
Step 3	Shared Raw Cleanup <code>clean_property_frame()</code> Adds <code>property_type</code> flag (flat/house), applies price normalization, drops source-only fields (<code>link</code> , <code>property_id</code>), and removes rows with missing target price.

Step 4	Frame Merging <code>merge_property_frames()</code> Concatenates cleaned flat and house DataFrames, aligns columns across different schemas, resets the index, and drops duplicate records.
Step 5	Level-Two Cleanup <code>clean_level_two()</code> Selects the curated modeling column set, normalizes remaining sector values, and drops any remaining duplicates.

Outlier Treatment

An `area_room_ratio` feature was computed to flag implausible listings: very large bedroom counts with tiny area, extreme area-per-room values, and price-per-sqft anomalies. Outlier treatment reduced the dataset from 3,803 to 3,555 rows, improving central-market prediction stability.

06 FEATURE ENGINEERING

Feature engineering in `src/features/engineering.py` and `src/features/selection.py` transforms semi-structured listing fields into 12 model-ready features.

Feature	Source Field	Engineering Method
agePossession	Raw text	6-bucket map: New Property / Relatively New / Moderately Old / Old Property / Under Construction / Undefined
built_up_area	areaWithType	Regex extraction + unit conversion: sq yard ×9, sq m ×10.764, acre ×43,560 → float sqft
furnishing_type	furnishDetails	Count parsed list items → unfurnished (0) / semifurnished (1–3) / furnished (4+)
luxury_score	features text	Weighted scan: pool(30), club(25), gym(20), security(15), park/lift/power/parking/garden(10 each)
luxury_category	luxury_score	Low (<50) / Medium (50–149) / High (≥150)
floor_category	floorNum	Low Floor (0–2) / Mid Floor (3–10) / High Floor (>10)
servant room	additionalRoom	Binary flag (0/1) — substring match in raw text
store room	additionalRoom	Binary flag (0/1) — substring match in raw text

Final 12 Model Features

CATEGORICAL FEATURES (OneHotEncoder)						
property_type	sector	balcony	agePossession	furnishing_type	luxury_category	floor_category
NUMERIC FEATURES (StandardScaler)						
bedRoom	bathroom	built_up_area	servant room	store room		

07 MACHINE LEARNING PIPELINE

The production training pipeline (`src/models/train.py`) builds a fully serialized sklearn pipeline encapsulating all preprocessing and model logic in a single joblib artifact. Command: `python -m src.models.train`

```
TransformedTargetRegressor(func=log1p, inverse_func=expm1)

### Pipeline

### preprocessor: ColumnTransformer

# ### categorical → OneHotEncoder(handle_unknown='ignore')

# ### numeric → StandardScaler()

### regressor: RandomForestRegressor(n_estimators=500, n_jobs=-1, random_state=42)
```

Configuration	Detail
Training data	gurgaon_properties_post_feature_selection_v2.csv (3,554 rows × 13 cols)
Train / test split	80% training / 20% held-out validation · random_state=42
Estimator	RandomForestRegressor · 500 trees · n_jobs=-1 (all available CPU cores)
Target transform	log1p applied before training · expm1 inverse applied to predictions
Categorical enc.	OneHotEncoder(handle_unknown='ignore') → unknown categories → all-zero vector
Numeric scaling	StandardScaler() · harmless for RF; preserves drop-in estimator compatibility
Artifact output	models/price_model.joblib (Git LFS tracked) + models/model_metadata.json
Metrics output	reports/metrics/model_metrics.json + reports/metrics/feature_importance.csv

Target Transformation Rationale

Real-estate prices are right-skewed — a small number of luxury properties dominate squared-error training and inflate RMSE. Training on **log1p(price)** compresses the target scale and makes relative prediction errors balanced across affordable, mid-range, and luxury segments. The transformation is serialized inside the sklearn artifact — inference automatically applies the inverse **expm1** transform, returning predictions in the original crore scale.

08 BACKEND DEVELOPMENT

The FastAPI backend (`app/api.py`) exposes five REST endpoints serving prediction and recommendation capabilities. Model and recommender are cached with `lru_cache(maxsize=1)`.

Method	Endpoint	Request Schema	Response
GET	/health	—	{'status': 'ok'}
POST	/predict	PropertyFeatures (12 fields + aliases)	{'predicted_price_crore': float}
POST	/batch_predict	{'records': [dict, ...]}	{'predicted_price_crore': [float, ...]}
POST	/recommend	{'query': str, 'top_n': int}	{'recommendations': [record, ...]}

POST	/similar_properties	{'property_name': str, 'top_n': int}	{'similar_properties': [record,...]}
------	---------------------	--------------------------------------	--------------------------------------

Prediction Request Lifecycle

1. User fills Streamlit form → prediction payload dictionary constructed with exact feature names
2. Frontend prediction_service.predict() → ApiClient.post('/predict', payload) via HTTP
3. FastAPI validates PropertyFeatures Pydantic schema — type check + alias resolution for spaced names
4. get_prediction_pipeline() returns lru_cached PriceInferencePipeline (loaded once at first call)
5. validate_frame() ensures all 12 MODEL_FEATURES present and correctly ordered before sklearn
6. OneHotEncoder → StandardScaler → RandomForestRegressor(500 trees) → expm1 inverse transform
7. JSON response {'predicted_price_crore': float} returned — Streamlit displays result card

09 FRONTEND DASHBOARD

The Streamlit frontend (**app/streamlit_app.py**) dispatches to four modular page renderers. All data loading is cached with `@st.cache_data`. Prediction and recommendation pages call the FastAPI backend via `ApiClient`.

Page	Key Features
Dashboard	Hero KPIs: listing count, median price, model R ² · Locality activity Mapbox map · Feature summary cards
Price Predictor	12-field property form → /predict → result card with nearby sector median comparison and price-per-sqft
Analytics	Property type + price range filters · Price-per-sqft Mapbox map · Sector heatmap · Distribution charts · Scatter plots
Recommendations	TF-IDF free-text query or similar-property lookup → enriched cards with estimated price range and top facilities

Confidence Range Note: The prediction confidence range in the UI uses a $\pm 12\%$ heuristic: $\text{spread} = \max(\text{value} \times 0.12, 0.15)$. This is a display convenience only — not a statistically calibrated interval. A future improvement would replace this with conformal prediction or quantile regression output.

10 ANALYTICS AND VISUALIZATION

The analytics layer (**src/analytics/insights.py** + **app/components/charts.py**) provides market intelligence through grouped sector aggregations and Plotly-based visualizations including Mapbox geographic maps.

Visualization	Business Question Answered
price_distribution	What is the overall spread of property prices across all listings?
area_price_scatter	How does built-up area relate to price, segmented by property type?
locality_trends	Which Gurgaon sectors command the highest median prices?
locality_heatmap	How do sector × property-type price medians compare across the market?
price_per_sqft_map	Which geographic areas are expensive per unit area (Mapbox scatter)?
locality_activity_map	Where is listing density geographically concentrated in Gurgaon?
feature_importance_chart	Which encoded features drive the RandomForest predictions most?

Geographic Analytics Pipeline

The `load_geo_market_data()` function joins sector-aggregated market data with `data/external/latlong.csv` (129 Gurgaon sector coordinates). It calculates $\text{price_per_sqft} = (\text{median_price} \times 10,000,000) / \text{median_area}$ for each sector, then powers Mapbox scatter maps displaying both listing density and price intensity geographically.

11 DEVOPS AND DEPLOYMENT

The system supports multiple deployment paths with a consistent Docker-based runtime. Git LFS tracks the model artifact (*.joblib) to prevent repository bloat.

Platform	Role	Configuration Notes
Railway	FastAPI backend	Production URL: gurgaon-real-estate-ml-production.up.railway.app · startup must use \$PORT
Streamlit Cloud	Frontend UI	.streamlit/config.toml present · reads Railway backend URL from session state
Docker	API container	FROM python:3.11-slim · EXPOSE 8000 · uvicorn app.api:api --host 0.0.0.0
Docker Compose	API + UI together	api:8000 + streamlit:8501 · same Dockerfile · ENVIRONMENT=docker env var
AWS	Cloud containers	ECR + ECS/Fargate or Elastic Beanstalk — documented in deployment/aws/README.md
GitHub Actions	CI/CD pipeline	push/PR to main → ruff lint → pytest → docker build validation

Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements ./requirements
RUN pip install --no-cache-dir -r requirements/api.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.api:api", "--host", "0.0.0.0", "--port", "8000"]
```

12 TESTING AND VALIDATION

Automated testing with `pytest` is integrated into the GitHub Actions CI pipeline. Tests run on every push and pull request to main, deliberately avoiding the large model artifact to keep CI fast.

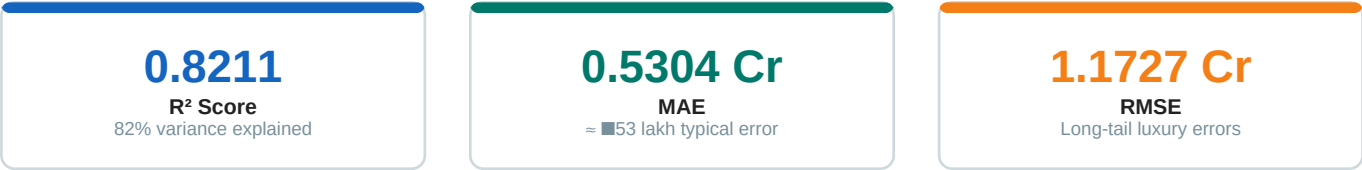
Test File	What Is Tested	Purpose
tests/test_api.py	GET /health → HTTP 200 → {'status': 'ok'}	Verifies FastAPI starts and responds correctly
tests/test_features.py	treat_price(), convert_to_sqft(), categorize_age_possession(), categorize_luxury(), categorize_floor()	Validates core feature transforms with known inputs and expected outputs

tests/test_inference_validation.py	Missing-feature ValueError; MODEL_FEATURES column ordering	Ensures inference schema catches bad inputs before sklearn processes them
------------------------------------	--	---

Recommended next step: add a full **/predict integration test** that loads the real price_model.joblib artifact in a staging environment and validates end-to-end prediction output against known property inputs with expected price ranges.

13 RESULTS ACHIEVED

The final model was evaluated on a 20% randomly held-out test set. All metrics are computed after inverse log transformation, expressed in the original crore scale.



R ² = 0.8211	Explains ~82% of price variance — strong for 12 structured features with no images or transaction dates. A random split may slightly overestimate performance if same-society listings appear in both train and test.
MAE = 0.5304 Cr	Typical absolute error is ~53 lakh rupees. Acceptable for decision support and portfolio analytics. May be material for affordable-segment buyers where total property cost is close to the error magnitude.
RMSE = 1.1727 Cr	RMSE being more than 2× MAE indicates a long-tail error distribution. Luxury villas and unusual properties cause the largest prediction misses. A separate luxury-segment model or quantile regression could reduce this.

System Capabilities Delivered

- Real-time single property price prediction via Streamlit form + FastAPI /predict endpoint
- Batch prediction support: multiple properties processed in a single API call via /batch_predict
- TF-IDF apartment recommendation engine across 247 Gurgaon projects via /recommend
- Similar property lookup using cosine similarity on project TF-IDF vectors via /similar_properties
- Locality analytics: sector median prices, heatmaps, area vs price scatter, locality trends
- Geospatial Mapbox visualizations using 129-point sector coordinate dataset from latlong.csv
- Production two-service deployment: Railway (FastAPI backend) + Streamlit Cloud (frontend)
- GitHub Actions CI: Ruff linting + pytest + Docker build validation on every pull request

14 CHALLENGES FACED

Several significant data and engineering challenges were encountered and resolved through systematic preprocessing, feature engineering, and architectural decisions.

Inconsistent Price Formats	Raw listings used 'Lac', 'Lakh', 'Cr', 'Crore' with mixed capitalizations and numeric formatting. Solved by regex-based normalization in treat_price() with standardized crore output.
----------------------------	--

Area Unit Diversity	Properties used square feet, yards, meters, and acres in the same areaWithType field. Solved by unit detection and conversion factors in convert_to_sqft().
Sparse Sector Categories	The same locality appeared as 'Sector 48 Gurgaon', 'sector 48', 'Sector 48, Gurugram'. Solved by normalize_sector() reducing fragmentation and improving per-category sample density.
Text-Heavy Amenity Fields	Amenities stored as Python list-like strings in CSV cells prevented direct parsing. Solved by ast.literal_eval with fallback comma-splitting in luxury_score() and get_furnishing_count().
Missing Area Values	Many listings provided only one of super built-up, built-up, or carpet area. Solved by extracting each separately and using built_up_area as the primary model feature.
Outlier Properties	Luxury villas and malformed listings created extreme price/area combinations damaging model quality. Solved by area_room_ratio outlier treatment, reducing the dataset from 3,803 to 3,555 rows.
Model Artifact Size	A RandomForest with 500 trees produces a large joblib file unsuitable for standard Git history. Solved by Git LFS tracking configured in .gitattributes for *.joblib files.
Unknown Categories at Inference	New sectors or property types not seen during training would crash the sklearn pipeline. Solved by OneHotEncoder(handle_unknown='ignore') silently mapping unknowns to zero-vectors.

15 SCALABILITY AND FUTURE IMPROVEMENTS

Although currently focused on Gurgaon, EstateIQ's modular FastAPI + Docker + CI/CD + reusable ML pipeline architecture provides a strong foundation for a multi-city enterprise real estate platform.

Known Technical Debt

- Raw→processed pipeline partly notebook-preserved, not fully scripted
- configs/model_config.yaml not loaded by training script
- MODEL_PATH env var not wired into PriceInferencePipeline
- /health does not validate model or recommender readiness
- Random split only — no temporal or locality-grouped validation
- ±12% confidence range is heuristic, not statistically calibrated
- No production logging, drift monitoring, or unknown-category tracking

Future Roadmap

- Script the complete raw→processed pipeline end-to-end
- Add Pandera / Pydantic DataFrame validation
- Implement conformal prediction or quantile regression intervals
- Add model versioning with metadata hashes + model registry
- Add /ready endpoint verifying model + recommender loaded
- Add drift monitoring: sector distribution, unknown-category rate
- Structured JSON logs + Prometheus / OpenTelemetry metrics
- Multi-city support with configurable per-city training pipelines

Long-Term Vision

Multi-City Support Configurable per-city preprocessing pipelines and model training workflows	Geospatial Filtering True geocoded radius filtering replacing text-only TF-IDF query approach
--	--

Semantic Search Dense text embeddings for deeper semantic recommendation matching	Personalization User interaction logs enabling collaborative filtering and learning-to-rank
Automated Retraining Scheduled pipelines triggered by data drift or listing volume changes	Deep Learning Models Gradient boosting or neural network alternatives with full hyperparameter tuning
Real-Time Data Integration Live property API feeds replacing static CSV-based datasets	Mobile Application React Native or Flutter front end consuming the production FastAPI backend